PRACTICAL RECORD NOTEBOOK

Programming Concepts in Mechanical Engineering

Program	MECHANICAL ENGG
Semester	III
Course name	Programming Concepts in Mechanical Engineering
Type of course	Integrated
Course code	25ME34I
Course coordinator	Vinay Kumar
Student name / USN	
Notebook style	College practical record (simple English)

Index of experiments

No.	Experiment
1(a)	Algorithm and flowchart: Convert Fahrenheit to Centigrade
1(b)	Algorithm and flowchart: Efficiency of a simple machine
1(c)	Algorithm and flowchart: Multiplication table of 5 (control loop)
2	Practice problems on type conversions and working with variables
3	Factorial of a number using a while loop
4	GCD of two numbers using a function
5	Student names in a list: add, remove and sort
6	Read data from a CSV file and count rows
7	Bank account class: deposit, withdrawal, balance inquiry
8	Motor driver circuit: controlling a DC motor with L298N and Raspberry Pi
9	Building a simple robot using Raspberry Pi, motors and sensors
10	Sample average and range; X-bar and R charts using Matplotlib
11	Mean, variance and standard deviation using Python
12	Quality control tools: plots for given data
13	Stress distribution along a plate (bar)

Notes for the student: (1) Experiment 1 contains only the algorithm and flowchart. Program outputs were captured by actually running the code (Python 3.12); the programs for Experiments 8 and 9 were run only as software tests, **not on hardware**. (2) Data marked 'sample' are illustrative, not laboratory measurements; enter your own readings where blank fields are provided. (3) GPIO pin numbers in Experiments 8 and 9 are assumed examples; match them to your actual wiring. (4) Flowchart symbols: rounded box = START/STOP, parallelogram = INPUT/OUTPUT, rectangle = PROCESS, diamond = DECISION.

EXPERIMENT 1(a): CONVERT FAHRENHEIT TO CENTIGRADE

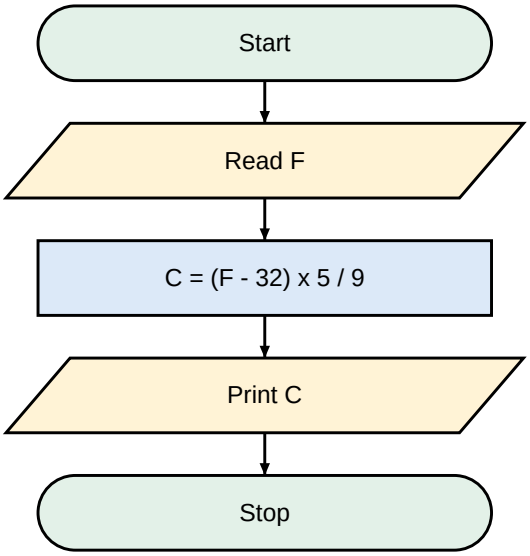
AIM

To write the algorithm and draw the flowchart to convert a temperature from Fahrenheit to Centigrade (Celsius).

ALGORITHM

1. Start.
2. Read the temperature F in Fahrenheit.
3. Calculate C = (F - 32) x 5 / 9.
4. Print C.
5. Stop.

FLOWCHART



EXPERIMENT 1(b): EFFICIENCY OF A SIMPLE MACHINE

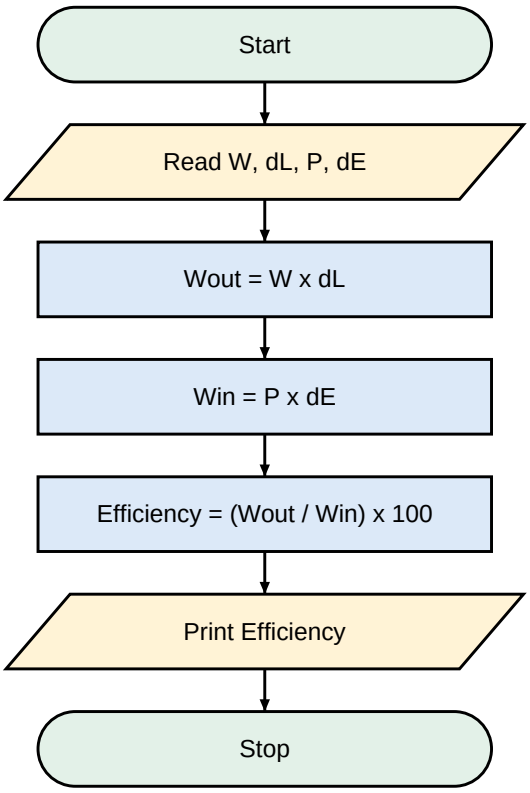
AIM

To write the algorithm and draw the flowchart to find the efficiency of a simple machine.

ALGORITHM

1. Start.
2. Read load W, load distance dL, effort P and effort distance dE.
3. Calculate Wout = W x dL.
4. Calculate Win = P x dE.
5. Calculate Efficiency = (Wout / Win) x 100.
6. Print Efficiency.
7. Stop.

FLOWCHART



EXPERIMENT 1(c): MULTIPLICATION TABLE OF 5 (CONTROL LOOP)

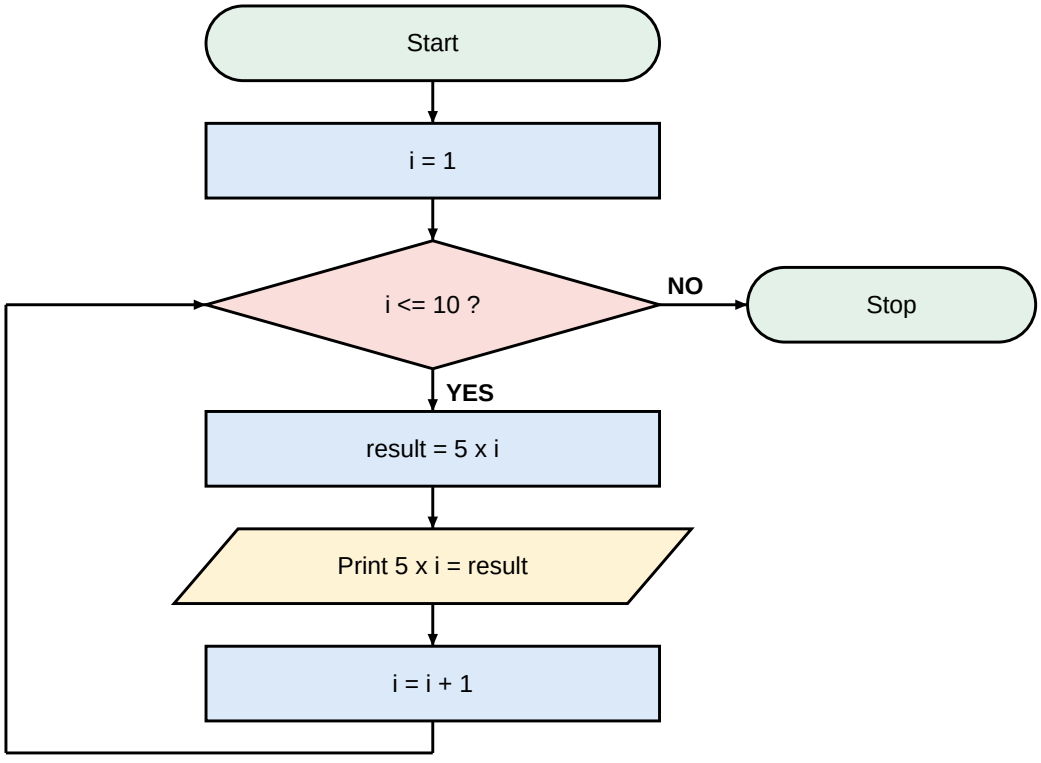
AIM

To write the algorithm and draw the flowchart to generate the multiplication table of 5 using a control loop.

ALGORITHM

1. Start.
2. Set i = 1.
3. Check i <= 10. If NO, go to Step 7.
4. Calculate result = 5 x i.
5. Print "5 x i = result".
6. Set i = i + 1 and go back to Step 3.
7. Stop.

FLOWCHART



A. EXPERIMENT 2: TYPE CONVERSIONS AND WORKING WITH VARIABLES

B. AIM / OBJECTIVE

To practise type conversions and working with variables in Python (int, float, str, bool).

C. ALGORITHM

1. Start.
2. Read name, age and height as text (str).
3. Show the type of each variable.
4. Convert age to int and height to float; use them in calculations.
5. Show int to float, float to int and rounding examples.
6. Convert a number to str and join it with text.
7. Show bool conversion of 0, 7, empty and non-empty strings.
8. Use arithmetic operators on two variables.
9. Swap two variables using a temporary variable.
10. Show the ValueError for an invalid conversion using try/except.
11. Stop.

D. PROGRAM

Python program with eight practice problems:

```
# Experiment 2: Type conversions and working with variables
name = input("Enter student name: ")
age_text = input("Enter age (years): ")
height_text = input("Enter height (m): ")

print("--- Problem 1: Data type of each variable ---")
print("name        :", type(name))
print("age_text    :", type(age_text))
print("height_text :", type(height_text))

print("--- Problem 2: str to int and str to float ---")
age = int(age_text)
height = float(height_text)
print("age + 1      =", age + 1, type(age))
print("height * 100 =", round(height * 100, 1), type(height))

print("--- Problem 3: int to float and float to int ---")
print("float(age)  =", float(age))
print("int(3.99)   =", int(3.99), "(decimal part is cut off)")
print("round(3.99) =", round(3.99))

print("--- Problem 4: number to str ---")
message = name + " is " + str(age) + " years old"
print(message)

print("--- Problem 5: bool conversion ---")
print("bool(0)   =", bool(0))
print("bool(7)   =", bool(7))
print("bool('')  =", bool(""))
print("bool('A') =", bool("A"))

print("--- Problem 6: Arithmetic on variables ---")
x = 7
y = 2
print("x / y  =", x / y)
print("x // y =", x // y)
print("x % y  =", x % y)
print("x ** y =", x ** y)
```

```
print("--- Problem 7: Swapping two variables ---")
a = 10
b = 20
print("Before swap: a =", a, ", b =", b)
temp = a
a = b
b = temp
print("After swap : a =", a, ", b =", b)

print("--- Problem 8: Invalid conversion ---")
try:
    int("abc")
except ValueError as error:
    print("ValueError:", error)
```

- **type(x)** shows the data type of x.
- **int(age_text)** and **float(height_text)** convert the keyboard text to numbers.
- **str(age)** is needed to join a number with text using +.
- **try ... except** catches the ValueError so the program does not crash.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Enter student name: Asha
Enter age (years): 20
Enter height (m): 1.65
--- Problem 1: Data type of each variable ---
name        : <class 'str'>
age_text    : <class 'str'>
height_text : <class 'str'>
--- Problem 2: str to int and str to float ---
age + 1      = 21 <class 'int'>
height * 100 = 165.0 <class 'float'>
--- Problem 3: int to float and float to int ---
float(age)  = 20.0
int(3.99)   = 3 (decimal part is cut off)
round(3.99) = 4
--- Problem 4: number to str ---
Asha is 20 years old
--- Problem 5: bool conversion ---
bool(0)   = False
bool(7)   = True
bool('')  = False
bool('A') = True
--- Problem 6: Arithmetic on variables ---
x / y  = 3.5
x // y = 3
x % y  = 1
x ** y = 49
--- Problem 7: Swapping two variables ---
Before swap: a = 10 , b = 20
After swap : a = 20 , b = 10
--- Problem 8: Invalid conversion ---
ValueError: invalid literal for int() with base 10: 'abc'
```

F. OBSERVATION

Expression	Result (sample run)	Type of result
int('20') + 1	21	int
float(20)	20.0	float
int(3.99)	3	int
round(3.99)	4	int
name + ' is ' + str(age)	Asha is 20 years old	str
bool(0), bool('')	False, False	bool
bool(7), bool('A')	True, True	bool
7 / 2	3.5	float
7 // 2	3	int
7 % 2	1	int
7 ** 2	49	int

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

Type conversions between int, float, str and bool, arithmetic operators and variable swapping were practised, and the program output matched the expected results (sample run).

A. EXPERIMENT 3: FACTORIAL OF A NUMBER USING A WHILE LOOP

B. AIM / OBJECTIVE

To write a Python program to calculate the factorial of a number using a while loop.

C. ALGORITHM

1. Start.
2. Read n (a non-negative integer).
3. Set fact = 1 and i = 1.
4. Check i <= n. If NO, go to Step 7.
5. Calculate fact = fact x i.
6. Set i = i + 1 and go back to Step 4.
7. Print fact.
8. Stop.

D. PROGRAM

Python program that follows the algorithm exactly (assumes n is a non-negative integer):

```
# Experiment 3: Factorial of a number using a while loop
n = int(input("Enter a non-negative integer n: "))
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
print(f"Factorial of {n} = {fact}")
```

- **fact = 1** is the starting product (multiplying by 1 changes nothing).
- **while i <= n** runs the loop n times; when n = 0 it does not run, so the answer stays 1.
- **fact = fact * i** updates the product; **i = i + 1** moves the counter.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Enter a non-negative integer n: 5
Factorial of 5 = 120
```

F. OBSERVATION

i	fact before	fact after (fact x i)
1	1	1
2	1	2
3	2	6
4	6	24
5	24	120

n (sample)	0	1	5	7
n! (program output)	1	1	120	5040

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The program calculated the factorial using a while loop. For n = 5 the output was 120 (sample run).

A. EXPERIMENT 4: GCD OF TWO NUMBERS USING A FUNCTION

B. AIM / OBJECTIVE

To create a Python program to find the greatest common divisor (GCD) of two numbers using a function.

C. ALGORITHM

1. **Function gcd(a, b):**
2. Start gcd. Take a = |a| and b = |b|.
3. Check b != 0. If NO, go to Step 6.
4. Calculate r = a mod b.
5. Set a = b and b = r, then go back to Step 3.
6. Return a. End of function.
7. **Main program:** Read x and y; call gcd(x, y); print the returned value.

D. PROGRAM

Python program using a user-defined function:

```
# Experiment 4: GCD of two numbers using a function (Euclid's method)
def gcd(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        r = a % b
        a = b
        b = r
    return a


x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
print(f"GCD of {x} and {y} = {gcd(x, y)}")
```

- **def gcd(a, b):** defines the function; **return a** sends the answer back to the caller.
- **abs()** makes the numbers positive so negative inputs also work.
- **%** gives the remainder; the loop stops when the remainder chain reaches 0.
- **gcd(x, y)** in the last line calls the function.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Enter first number: 48
Enter second number: 18
GCD of 48 and 18 = 6
```

F. OBSERVATION

Pass	a	b	r = a mod b
1	48	18	12
2	18	12	6
3	12	6	0
End	6	0	- (loop stops, return 6)

x (sample)	y (sample)	GCD (program output)
48	18	6
100	75	25
17	5	1
54	24	6

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The program found the GCD using a function and Euclid's method. For 48 and 18 the output was 6 (sample run).

A. EXPERIMENT 5: STUDENT NAMES IN A LIST: ADD, REMOVE AND SORT

B. AIM / OBJECTIVE

To write a Python program that stores the names of students in a list and performs the operations add, remove and sort.

C. ALGORITHM

1. Start.
2. Create the list students with four names.
3. Read the name to add (new_name).
4. Add new_name to the end of the list.
5. Read the name to remove (remove_name).
6. Check whether remove_name is in the list. If YES, remove it; if NO, print 'not in the list'.
7. Sort the list from A to Z and print it.
8. Sort the list from Z to A and print it.
9. Print the total number of students.
10. Stop.

D. PROGRAM

Python program that follows the algorithm exactly:

```
# Experiment 5: Student names in a list - add, remove and sort
students = ["Rahul", "Meena", "Arjun", "Divya"]
print("Initial list   :", students)

new_name = input("Enter a name to ADD: ")
students.append(new_name)
print("After adding   :", students)

remove_name = input("Enter a name to REMOVE: ")
if remove_name in students:
    students.remove(remove_name)
    print("After removing :", students)
else:
    print(remove_name, "is not in the list")

students.sort()
print("Sorted (A to Z):", students)
students.sort(reverse=True)
print("Sorted (Z to A):", students)
print("Total students =", len(students))
```

• **append()** adds the new name at the end.
• **if remove_name in students** avoids the ValueError that remove() would raise for a missing name.
• **sort()** changes the list itself (in place); **reverse=True** gives Z to A.
• **len()** counts the items.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Initial list   : ['Rahul', 'Meena', 'Arjun', 'Divya']
Enter a name to ADD: Kiran
After adding   : ['Rahul', 'Meena', 'Arjun', 'Divya', 'Kiran']
Enter a name to REMOVE: Meena
After removing : ['Rahul', 'Arjun', 'Divya', 'Kiran']
Sorted (A to Z): ['Arjun', 'Divya', 'Kiran', 'Rahul']
Sorted (Z to A): ['Rahul', 'Kiran', 'Divya', 'Arjun']
Total students = 4
```

F. OBSERVATION

Step	Operation	List after the operation (sample run)	Count
1	Initial list	Rahul, Meena, Arjun, Divya	4
2	Add Kiran	Rahul, Meena, Arjun, Divya, Kiran	5
3	Remove Meena	Rahul, Arjun, Divya, Kiran	4
4	Sort A to Z	Arjun, Divya, Kiran, Rahul	4
5	Sort Z to A	Rahul, Kiran, Divya, Arjun	4

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The program stored names in a list and performed add, remove and sort operations correctly. The final A-Z list for the sample run was Arjun, Divya, Kiran, Rahul.

A. EXPERIMENT 6: READING A CSV FILE AND COUNTING ROWS

B. AIM / OBJECTIVE

To write a Python program to read data from a CSV file and perform basic operations such as counting the number of rows.

C. ALGORITHM

1. Start.
2. Open the file materials.csv and read the header (column names).
3. Set count = 0 and total = 0.
4. Check whether there is one more data row. If NO, go to Step 7.
5. Add 1 to count.
6. Convert Yield_MPa of the row to a number and add it to total. Go back to Step 4.
7. Print the number of columns, number of data rows (count) and average = total / count.
8. Stop.

D. PROGRAM

Python program that follows the algorithm exactly. Save the CSV data given in Section E as materials.csv in the same folder before running:

```
# Experiment 6: Read a CSV file and perform basic operations
import csv

count = 0
total = 0.0
with open("materials.csv", newline="") as file:
    reader = csv.DictReader(file)
    print("Column names:", reader.fieldnames)
    print("Number of columns =", len(reader.fieldnames))
    for row in reader:
        count = count + 1
        total = total + float(row["Yield_MPa"])

print("Number of data rows =", count)
print(f"Average yield strength = {total / count:.1f} MPa")
```

- **csv.DictReader(file)** reads rows as dictionaries; **reader.fieldnames** gives the header.
- **for row in reader** visits each data row (the header is skipped automatically).
- **float(row['Yield_MPa'])** converts text to a number.
- **newline=''** is the recommended setting when opening CSV files.

E. INPUT / TEST DATA

Sample dataset - file **materials.csv** (illustrative values, not measured data; use your own file if one is given):

```
ID,Material,Density_kg_m3,Yield_MPa
1,Mild Steel,7850,250
2,Aluminium 6061,2700,275
3,Copper,8960,70
4,Brass,8500,200
5,Grey Cast Iron,7200,130
```

F. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Column names: ['ID', 'Material', 'Density_kg_m3', 'Yield_MPa']
Number of columns = 4
Number of data rows = 5
Average yield strength = 185.0 MPa
```

G. OBSERVATION

Item	Value (sample run)
Lines in file (including header)	6
Number of columns	4
Number of data rows (count)	5
Total of Yield_MPa	925
Average Yield_MPa	185.0 MPa

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

H. RESULT

The program read the CSV file, found 4 columns and 5 data rows, and calculated an average yield strength of 185.0 MPa for the sample file.

A. EXPERIMENT 7: BANK ACCOUNT CLASS

B. AIM / OBJECTIVE

To develop a Python class for a bank account that supports deposit, withdrawal and balance inquiry.

C. ALGORITHM

1. Start. Define class BankAccount with owner and balance (constructor).
2. **deposit(amount):** if amount > 0, balance = balance + amount and print the new balance; otherwise print an error.
3. **withdraw(amount):** if 0 < amount <= balance, balance = balance - amount and print the new balance; otherwise print an error.
4. **get_balance():** return balance.
5. Main: create the account for Asha with balance 1000.
6. Call deposit(500), withdraw(300), withdraw(5000), deposit(-50).
7. Print the balance inquiry. Stop.

D. PROGRAM

Python program with the BankAccount class and a demonstration:

```
# Experiment 7: Bank account class with deposit, withdrawal and balance inquiry
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance = self.balance + amount
            print(f"Deposited {amount:.2f}. New balance = {self.balance:.2f}")
        else:
            print("Deposit failed: amount must be positive")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance = self.balance - amount
            print(f"Withdrew {amount:.2f}. New balance = {self.balance:.2f}")
        else:
            print("Withdrawal failed: invalid amount or insufficient balance")

    def get_balance(self):
        return self.balance


account = BankAccount("Asha", 1000.0)
print("Account holder:", account.owner)
print(f"Opening balance = {account.get_balance():.2f}")
account.deposit(500)
account.withdraw(300)
account.withdraw(5000)
account.deposit(-50)
print(f"Balance inquiry = {account.get_balance():.2f}")
```

- **class BankAccount:** defines the class; **__init__** stores owner and balance (default 0.0).
- **self.balance** is the balance of that particular account object.
- **0 < amount <= self.balance** checks both positive amount and sufficient balance in one test.
- **get_balance()** returns the value so it can be printed or used.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Account holder: Asha
Opening balance = 1000.00
```

```
Deposited 500.00. New balance = 1500.00
Withdrew 300.00. New balance = 1200.00
Withdrawal failed: invalid amount or insufficient balance
Deposit failed: amount must be positive
Balance inquiry = 1200.00
```

F. OBSERVATION

S.No.	Operation	Amount	Balance before	Balance after	Status
1	Deposit	500	1000.00	1500.00	Success
2	Withdraw	300	1500.00	1200.00	Success
3	Withdraw	5000	1200.00	1200.00	Failed (insufficient balance)
4	Deposit	-50	1200.00	1200.00	Failed (invalid amount)
5	Balance inquiry	-	1200.00	1200.00	Shown

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The BankAccount class supported deposit, withdrawal and balance inquiry. For the sample test the final balance was 1200.00, and invalid transactions were rejected.

A. EXPERIMENT 8: MOTOR DRIVER CIRCUIT: DC MOTOR WITH L298N AND RASPBERRY PI

B. AIM / OBJECTIVE

To control the speed and direction of a DC motor using an L298N motor driver and a Raspberry Pi.

C. ALGORITHM

1. Start.
2. Import Motor from gpiozero and the sleep function.
3. Create the motor object using the assumed pins IN1, IN2 and ENA with PWM.
4. Run the motor forward at 50% speed for 3 s, then stop for 1 s.
5. Run the motor forward at 100% speed for 3 s, then stop for 1 s.
6. Run the motor backward at 50% speed for 3 s, then stop for 1 s.
7. Stop the motor and print 'Cleanup done'.
8. Stop.

D. PROGRAM / CIRCUIT DESIGN

Assumptions (please confirm or correct with your lab hardware):

- Raspberry Pi with the standard 40-pin header, running Raspberry Pi OS with Python 3 and the **gpiozero** library.
- L298N dual H-bridge motor driver module (red/blue board with ENA/ENB jumpers) and **one** brushed DC motor connected to channel A.
- A separate battery or DC supply for the motor, chosen according to the motor rating and the L298N module label.
- The GPIO numbers (BCM numbering) used below are **assumed examples**. Use any free GPIO pins, but the program and wiring must match.

Connection table (channel A):

L298N terminal	Connect to	Note	Your actual wiring
OUT1, OUT2	Two terminals of the DC motor	Swapping the two motor wires reverses the rotation	
12V (motor supply +)	Positive of the external motor supply	Voltage must suit the motor and the module	
GND	Negative of motor supply **and** a Raspberry Pi GND pin (e.g. physical pin 6)	Common ground is essential	
5V terminal	Leave unconnected	Do not power the Raspberry Pi from this pin	
ENA	GPIO22 (assumed), physical pin 15	**Remove the ENA jumper** so PWM can control speed	
IN1	GPIO17 (assumed), physical pin 11	Direction input 1	
IN2	GPIO27 (assumed), physical pin 13	Direction input 2	

Working of the main components:

- **Raspberry Pi:** sends logic signals (3.3 V) but cannot supply motor current, so a driver is needed.
- **L298N H-bridge:** uses the logic signals to connect the motor to the motor supply in either polarity, so the motor can run forward or backward.
- **ENA with PWM:** pulse-width modulation switches the motor supply on and off very quickly; a higher duty cycle gives a higher average voltage and more speed.
- **External supply:** provides the motor current.

Direction truth table (ENA = HIGH):

IN1	IN2	Motor
HIGH	LOW	Rotates in one direction (forward)
LOW	HIGH	Rotates in the opposite direction (backward)
LOW	LOW	Stops (brake)
HIGH	HIGH	Stops (brake)

Safe operating procedure:

1. Switch OFF all power. Make the connections in the table and check them twice.
2. Confirm the common ground between the Pi and the L298N, and that the ENA jumper is removed.
3. Fix the motor so it cannot move or fall; keep fingers and wires away from the shaft.
4. Power the Raspberry Pi and run the program. Then switch ON the motor supply.
5. Watch the motor: it should run forward at 50% speed, stop, run forward at full speed, stop, then run backward at 50% speed.
6. Press Ctrl+C to stop at any time. Switch OFF the motor supply first, then shut down the Pi.

Troubleshooting:

Problem	Likely cause	Check
Motor does not move	Motor supply off, or no common ground	Supply switch, GND wires
Motor runs at full speed only	ENA jumper still fitted	Remove the jumper; connect ENA to the Pi
Direction is opposite to expected	Motor wires swapped	Swap OUT1/OUT2 or IN1/IN2 in the program
Pi restarts when the motor starts	Power problem or shared supply	Use separate supplies with common ground
L298N gets very hot	Motor current too high	Check motor rating; use a heat sink or smaller load

Missing details that could change the wiring or code: Raspberry Pi model, motor voltage and current rating, power-supply type, L298N module version, and the GPIO pins actually used.

Program (the GPIO numbers are assumed examples):

```
# Experiment 8: DC motor control with L298N and Raspberry Pi (gpiozero library)
# WARNING: the GPIO numbers below (BCM numbering) are ASSUMED EXAMPLES.
# Change them to match YOUR actual wiring before running on hardware.
from time import sleep
from gpiozero import Motor

IN1 = 17    # assumed: Raspberry Pi GPIO17 -> L298N IN1
IN2 = 27    # assumed: Raspberry Pi GPIO27 -> L298N IN2
ENA = 22    # assumed: Raspberry Pi GPIO22 -> L298N ENA (jumper removed)

motor = Motor(forward=IN1, backward=IN2, enable=ENA, pwm=True)


def run(direction, speed, seconds):
    if direction == "forward":
        motor.forward(speed)
    else:
        motor.backward(speed)
    print(f"Motor {direction} at {int(speed * 100)} % for {seconds} s")
    sleep(seconds)
    motor.stop()
    print("Motor stopped")
    sleep(1)


try:
    run("forward", 0.5, 3)
    run("forward", 1.0, 3)
    run("backward", 0.5, 3)
```

```
finally:
    motor.stop()
    print("Cleanup done")
```

- **Motor(forward=IN1, backward=IN2, enable=ENA, pwm=True)** creates a motor object with PWM speed control.
- **forward(speed)** and **backward(speed)** take a value from 0 to 1 (0.5 = 50%).
- **stop()** switches the motor off.
- **try ... finally** makes sure the motor is stopped even if the program is interrupted.

E. OUTPUT / EXPECTED OUTPUT

Console output of a software-only dry run using gpiozero's mock pins (no motor was connected). On real hardware the same messages appear while the motor moves.

```
Motor forward at 50 % for 3 s
Motor stopped
Motor forward at 100 % for 3 s
Motor stopped
Motor backward at 50 % for 3 s
Motor stopped
Cleanup done
```

Expected physical behaviour (to be confirmed on the real set-up): the motor turns in one direction at medium speed for 3 s, stops for 1 s, turns faster for 3 s, stops for 1 s, then turns in the opposite direction at medium speed for 3 s and stops.

F. OBSERVATION

Test	Direction	Speed set (%)	Time (s)	Observed behaviour (student to fill)	Motor speed / RPM (optional)
1	Forward	50	3		
2	Forward	100	3		
3	Backward	50	3		

No actual hardware readings are included here. The student must enter the real observations after testing.

G. RESULT

The program and wiring plan for controlling a DC motor with the L298N and Raspberry Pi were prepared. Hardware performance must be confirmed by the student from actual observations; the software dry run printed the expected messages.

A. EXPERIMENT 9: SIMPLE ROBOT USING RASPBERRY PI, MOTORS AND SENSORS

B. AIM / OBJECTIVE

To build a simple obstacle-avoiding robot using a Raspberry Pi, DC motors with an L298N driver and an ultrasonic sensor.

C. ALGORITHM

1. Start.
2. Set STOP_DISTANCE_CM = 20 and SPEED = 0.6. Set up both motors and the sensor.
3. Read the distance d in cm.
4. If d > 20 cm, run both motors forward.
5. Otherwise stop, reverse for 0.5 s, then turn right for 0.6 s (left motor forward, right motor backward).
6. Wait 0.1 s and go back to Step 3.
7. When Ctrl+C is pressed, stop both motors and end.

D. PROGRAM / CIRCUIT DESIGN

Assumptions (please confirm or correct with your lab hardware):

- Two-wheel robot chassis with two brushed DC geared motors (left and right) and a caster wheel.
- L298N motor driver module, Raspberry Pi (40-pin header) with Raspberry Pi OS, Python 3 and gpiozero.
- **One HC-SR04 ultrasonic distance sensor** is chosen as the sensor (an assumption; other sensors need changes).
- Separate supplies: a battery pack for the motors (suited to the motors and L298N) and a power bank for the Raspberry Pi, with a common ground.
- All GPIO numbers (BCM numbering) below are **assumed examples**.

Connection table:

From	To	Note	Your wiring
L298N OUT1, OUT2	Left motor	Swap wires if the wheel spins the wrong way	
L298N OUT3, OUT4	Right motor	Same as above	
L298N 12V terminal	Positive of motor battery	Check motor and module ratings	
L298N GND	Battery negative and Pi GND	Common ground	
L298N IN1, IN2, ENA	GPIO17, GPIO27, GPIO22 (assumed)	Left motor; remove ENA jumper	
L298N IN3, IN4, ENB	GPIO23, GPIO24, GPIO25 (assumed)	Right motor; remove ENB jumper	
HC-SR04 VCC	Pi 5V (physical pin 2)	Sensor needs 5 V	
HC-SR04 GND	Pi GND		
HC-SR04 TRIG	GPIO5 (assumed)	Trigger input	
HC-SR04 ECHO	Voltage divider, then GPIO6 (assumed)	ECHO gives 5 V; the Pi accepts only 3.3 V	

ECHO voltage divider: ECHO → 1 kΩ resistor → GPIO6 node → 2 kΩ resistor → GND. The GPIO pin is connected at the junction, giving about 5 x 2 / (1 + 2) = 3.3 V.

Working of the robot:

- The HC-SR04 sends an ultrasonic pulse and measures the echo time; the Pi converts it to distance.
- If the distance is more than 20 cm, both motors run forward.
- If the distance is 20 cm or less, the robot stops, reverses a little and turns right by running the left motor

forward and the right motor backward.

- The decision function is separate from the hardware, so it can be tested without the robot (python3 e9.py test).

Safe operating procedure:

1. Switch OFF all power. Make the connections and check them twice (especially the ECHO divider and common ground).
2. Lift the robot so the wheels are free. Power the Pi and start the program.
3. Switch ON the motor battery and check that both wheels turn forward. Fix wiring if a wheel turns backward.
4. Place the robot on the floor in an open area. Put an object about 15 cm in front of it and watch it back up and turn.
5. Press Ctrl+C to stop; switch OFF the motor battery first.

Troubleshooting:

Problem	Likely cause	Check
Distance always 200 cm or 0	TRIG/ECHO wires wrong or no 5 V	Wiring, divider and power
Robot goes in circles	One motor reversed	Swap that motor's wires
Robot does not stop in time	Speed too high or limit too small	Reduce SPEED or increase STOP_DISTANCE_CM
Pi resets when motors start	Shared or weak supply	Use separate supplies with common ground

Missing details that could change the design: Raspberry Pi model, chassis and motor ratings, battery type, which sensors are actually available (ultrasonic, IR, line sensor), and the GPIO pins used.

Program (GPIO numbers are assumed examples):

```
# Experiment 9: Obstacle-avoiding robot (Raspberry Pi + L298N + 2 DC motors + HC-SR04)
# WARNING: all GPIO numbers (BCM numbering) are ASSUMED EXAMPLES. Match them to YOUR wiring.
# Run   : python3 e9.py        (on the robot)
# Test  : python3 e9.py test   (logic check only, no hardware needed)
import sys
from time import sleep

STOP_DISTANCE_CM = 20     # obstacle distance limit
SPEED = 0.6               # 60 % motor speed

# Assumed pins
IN1, IN2, ENA = 17, 27, 22        # left motor
IN3, IN4, ENB = 23, 24, 25        # right motor
TRIG, ECHO = 5, 6                 # HC-SR04 (ECHO must go through a voltage divider)


def decide(distance_cm):
    if distance_cm > STOP_DISTANCE_CM:
        return "forward"
    return "avoid"


def run_robot():
    from gpiozero import Motor, DistanceSensor
    left = Motor(forward=IN1, backward=IN2, enable=ENA, pwm=True)
    right = Motor(forward=IN3, backward=IN4, enable=ENB, pwm=True)
    sensor = DistanceSensor(echo=ECHO, trigger=TRIG, max_distance=2.0)
    try:
        while True:
            distance = sensor.distance * 100          # metres to cm
            action = decide(distance)
            print(f"Distance = {distance:.0f} cm -> {action}")
            if action == "forward":
                left.forward(SPEED)
                right.forward(SPEED)
            else:
                left.stop()
```

```
                right.stop()
                sleep(0.2)
                left.backward(SPEED)                  # reverse a little
                right.backward(SPEED)
                sleep(0.5)
                left.forward(SPEED)                   # turn right on the spot
                right.backward(SPEED)
                sleep(0.6)
            sleep(0.1)
    except KeyboardInterrupt:
        print("Stopped by user")
    finally:
        left.stop()
        right.stop()


def test_logic():
    for d in [100, 45, 21, 20, 12, 5]:
        print(f"Distance = {d} cm -> {decide(d)}")


if len(sys.argv) > 1 and sys.argv[1] == "test":
    test_logic()
else:
    run_robot()
```

- **decide(distance_cm)** returns 'forward' or 'avoid'; it can be tested without hardware.
- **DistanceSensor(echo, trigger)** measures distance in metres; the program multiplies by 100 to get cm.
- **left.forward()** / **right.backward()** together make the robot turn on the spot.
- **try/except/finally** stops both motors when the program ends.

E. OUTPUT / EXPECTED OUTPUT

Output of the logic test 'python3 e9.py test'. This is a software-only check; no robot hardware was used.

```
Distance = 100 cm -> forward
Distance = 45 cm -> forward
Distance = 21 cm -> forward
Distance = 20 cm -> avoid
Distance = 12 cm -> avoid
Distance = 5 cm -> avoid
```

Expected physical behaviour (to be confirmed on the real robot): the robot moves forward while the path is clear. When an object is 20 cm or closer, it stops, backs up for about half a second, turns right for about 0.6 s and then moves forward again. While running, it prints lines such as 'Distance = 85 cm -> forward'.

F. OBSERVATION

Test	Obstacle distance (cm)	Expected action (program logic)	Actual behaviour (student to fill)
1	100	forward	
2	45	forward	
3	21	forward	
4	20	avoid	
5	12	avoid	
6	5	avoid	

The 'expected action' column comes from the logic test. The last column must be filled from real tests on the robot.

G. RESULT

The robot design, wiring plan and obstacle-avoidance program were prepared. The decision logic gave

'forward' for distances above 20 cm and 'avoid' for 20 cm or less. Actual robot performance must be confirmed by the student after testing on hardware.

A. EXPERIMENT 10: SAMPLE AVERAGE AND RANGE; X-BAR AND R CHARTS

B. AIM / OBJECTIVE

To calculate the sample average (X-bar) and sample range (R) for the given data and to plot X-bar and R control charts using Matplotlib.

C. ALGORITHM

1. Start.
2. Enter the k = 10 subgroups of n = 5 readings and the constants A2 = 0.577, D3 = 0, D4 = 2.114.
3. For each subgroup, calculate the average X-bar and the range R.
4. Calculate X-double-bar and R-bar.
5. Calculate UCL, CL and LCL for the X-bar chart and for the R chart.
6. Print the table of X-bar and R and the control limits; list any points outside the limits.
7. Plot the X-bar chart and the R chart with CL, UCL and LCL lines; save the figure.
8. Stop.

D. PROGRAM

Python program (requires Matplotlib):

```
# Experiment 10: X-bar and R charts using Matplotlib
import matplotlib.pyplot as plt

# 10 subgroups, each of size n = 5 (shaft diameter in mm) - SAMPLE data
data = [
    [25.02, 25.00, 24.99, 25.01, 25.01],
    [24.99, 25.01, 25.02, 25.00, 24.98],
    [25.01, 25.00, 25.02, 24.97, 24.99],
    [24.98, 24.97, 25.00, 25.01, 24.99],
    [25.03, 25.02, 25.01, 25.01, 25.02],
    [24.97, 25.01, 25.01, 24.96, 25.01],
    [24.99, 25.02, 24.99, 25.00, 25.01],
    [24.98, 25.01, 25.00, 25.01, 24.99],
    [25.02, 25.01, 25.00, 25.01, 25.03],
    [25.04, 24.97, 25.02, 25.01, 25.00],
]
A2, D3, D4 = 0.577, 0.0, 2.114        # control chart constants for n = 5

xbar = [sum(s) / len(s) for s in data]        # sample average of each subgroup
rng = [max(s) - min(s) for s in data]         # sample range of each subgroup
xbarbar = sum(xbar) / len(xbar)               # grand average
rbar = sum(rng) / len(rng)                    # average range

ucl_x = xbarbar + A2 * rbar
lcl_x = xbarbar - A2 * rbar
ucl_r = D4 * rbar
lcl_r = D3 * rbar

print("Subgroup   Xbar(mm)   R(mm)")
for i in range(len(data)):
    print(f"{i + 1:5d}     {xbar[i]:8.3f}   {rng[i]:6.2f}")
print(f"Grand average (X double bar) = {xbarbar:.4f} mm")
print(f"Average range (R bar)        = {rbar:.4f} mm")
print(f"X-bar chart: UCL = {ucl_x:.4f}, CL = {xbarbar:.4f}, LCL = {lcl_x:.4f}")
print(f"R chart    : UCL = {ucl_r:.4f}, CL = {rbar:.4f}, LCL = {lcl_r:.4f}")

out_x = [i + 1 for i, v in enumerate(xbar) if v > ucl_x or v < lcl_x]
out_r = [i + 1 for i, v in enumerate(rng) if v > ucl_r or v < lcl_r]
print("Subgroups outside X-bar limits:", out_x if out_x else "none")
print("Subgroups outside R limits    :", out_r if out_r else "none")

k = list(range(1, len(data) + 1))
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 7))
ax1.plot(k, xbar, marker="o", color="tab:blue", label="Sample average")
ax1.axhline(xbarbar, color="green", label="CL")
ax1.axhline(ucl_x, color="red", linestyle="--", label="UCL")
ax1.axhline(lcl_x, color="red", linestyle="--", label="LCL")
ax1.set_title("X-bar Chart")
ax1.set_xlabel("Subgroup number")
ax1.set_ylabel("Sample average (mm)")
ax1.legend()
ax2.plot(k, rng, marker="s", color="tab:orange", label="Sample range")
ax2.axhline(rbar, color="green", label="CL")
ax2.axhline(ucl_r, color="red", linestyle="--", label="UCL")
ax2.axhline(lcl_r, color="red", linestyle="--", label="LCL")
ax2.set_title("R Chart")
ax2.set_xlabel("Subgroup number")
ax2.set_ylabel("Sample range (mm)")
ax2.legend()
plt.tight_layout()
plt.savefig("xbar_r_chart.png", dpi=150)
plt.show()
```

- The list **data** holds 10 subgroups, each with 5 readings.
- **max(s) - min(s)** gives the range of a subgroup.
- **plt.axhline()** draws the CL, UCL and LCL lines.
- **plt.savefig()** saves the chart as an image; **plt.show()** displays it.

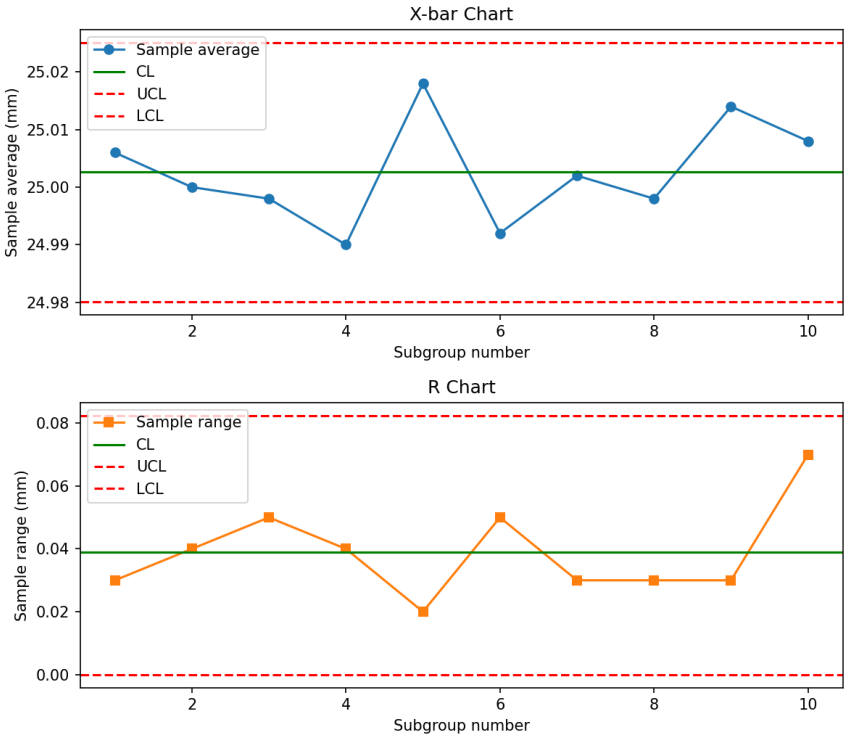
E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Subgroup   Xbar(mm)   R(mm)
    1       25.006     0.03
    2       25.000     0.04
    3       24.998     0.05
    4       24.990     0.04
    5       25.018     0.02
    6       24.992     0.05
    7       25.002     0.03
    8       24.998     0.03
    9       25.014     0.03
   10       25.008     0.07
Grand average (X double bar) = 25.0026 mm
Average range (R bar)        = 0.0390 mm
X-bar chart: UCL = 25.0251, CL = 25.0026, LCL = 24.9801
R chart    : UCL = 0.0824, CL = 0.0390, LCL = 0.0000
Subgroups outside X-bar limits: none
Subgroups outside R limits    : none
```

Expected graph: two charts one above the other. The upper X-bar chart shows 10 points joined by lines around the centre line 25.0026 mm between LCL 24.9801 mm and UCL 25.0251 mm. The lower R chart shows 10 range points around 0.039 mm with UCL 0.0824 mm and LCL 0. X-axis: subgroup number. Y-axis: average or range in mm.

Generated chart (sample data):



F. OBSERVATION

Subgroup	x1	x2	x3	x4	x5	X-bar (mm)	R (mm)
1	25.02	25.00	24.99	25.01	25.01	25.006	0.03
2	24.99	25.01	25.02	25.00	24.98	25.000	0.04
3	25.01	25.00	25.02	24.97	24.99	24.998	0.05
4	24.98	24.97	25.00	25.01	24.99	24.990	0.04
5	25.03	25.02	25.01	25.01	25.02	25.018	0.02
6	24.97	25.01	25.01	24.96	25.01	24.992	0.05
7	24.99	25.02	24.99	25.00	25.01	25.002	0.03
8	24.98	25.01	25.00	25.01	24.99	24.998	0.03
9	25.02	25.01	25.00	25.01	25.03	25.014	0.03
10	25.04	24.97	25.02	25.01	25.00	25.008	0.07

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The sample averages and ranges of 10 subgroups were calculated and X-bar and R charts were plotted. For the sample data X-double-bar = 25.0026 mm, R-bar = 0.039 mm, and all points were within the control limits.

A. EXPERIMENT 11: MEAN, VARIANCE AND STANDARD DEVIATION

B. AIM / OBJECTIVE

To calculate the mean, variance and standard deviation of a data set using Python.

C. ALGORITHM

1. Start.
2. Read the data list and find n = number of values.
3. Calculate mean = sum of values / n.
4. Calculate SS = sum of (x - mean) squared.
5. Calculate sample variance = SS / (n - 1) and sample standard deviation = square root of it.
6. Calculate population variance = SS / n and population standard deviation.
7. Print all results and verify with the statistics module.
8. Stop.

D. PROGRAM

Python program with manual calculation and a cross-check:

```
# Experiment 11: Mean, variance and standard deviation
import math
import statistics

# Tool life of 8 cutting tools in hours - SAMPLE data
data = [12, 15, 11, 18, 14, 16, 13, 17]
n = len(data)

mean = sum(data) / n
sum_sq = sum((x - mean) ** 2 for x in data)     # sum of squared deviations
var_sample = sum_sq / (n - 1)
var_pop = sum_sq / n
sd_sample = math.sqrt(var_sample)
sd_pop = math.sqrt(var_pop)

print("Data =", data)
print("n =", n)
print(f"Mean                          = {mean:.3f}")
print(f"Sum of squared deviations     = {sum_sq:.3f}")
print(f"Sample variance (n - 1)       = {var_sample:.3f}")
print(f"Sample standard deviation     = {sd_sample:.3f}")
print(f"Population variance (n)       = {var_pop:.3f}")
print(f"Population standard deviation = {sd_pop:.3f}")

print("Check with the statistics module:")
print(f"mean = {statistics.mean(data):.3f}, variance = {statistics.variance(data):.3f}, stdev = {statisti
print(f"pvariance = {statistics.pvariance(data):.3f}, pstdev = {statistics.pstdev(data):.3f}")
```

- **sum(data) / n** gives the mean.
- The generator **(x - mean) ** 2 for x in data** finds each squared deviation; **sum()** adds them.
- **math.sqrt()** gives the standard deviation.
- The last two lines confirm the results with the statistics module.

E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Data = [12, 15, 11, 18, 14, 16, 13, 17]
n = 8
Mean                          = 14.500
Sum of squared deviations     = 42.000
Sample variance (n - 1)       = 6.000
Sample standard deviation     = 2.449
```

```
Population variance (n)       = 5.250
Population standard deviation = 2.291
Check with the statistics module:
mean = 14.500, variance = 6.000, stdev = 2.449
pvariance = 5.250, pstdev = 2.291
```

F. OBSERVATION

S.No.	x (hours)	x - mean	$(x - mean)^2$
1	12	-2.5	6.25
2	15	0.5	0.25
3	11	-3.5	12.25
4	18	3.5	12.25
5	14	-0.5	0.25
6	16	1.5	2.25
7	13	-1.5	2.25
8	17	2.5	6.25
Total	116	0	42.00

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

For the sample data, mean = 14.500, sample variance = 6.000, sample standard deviation = 2.449, population variance = 5.250 and population standard deviation = 2.291. The statistics module gave the same values.

A. EXPERIMENT 12: QUALITY CONTROL TOOLS

B. AIM / OBJECTIVE

To write Python code to plot different quality control tools (Pareto chart, histogram, scatter diagram and control chart) for the given data.

C. ALGORITHM

1. Start.
2. Enter the defect counts, 30 diameter readings, and the speed-roughness data.
3. Pareto: arrange counts in descending order, calculate cumulative % and plot bars with the line.
4. Histogram: calculate mean and standard deviation of the diameters and plot the histogram.
5. Scatter: calculate the best-fit line and r, then plot points and line.
6. Control chart: calculate MR-bar, sigma, CL, UCL, LCL and plot the readings in order.
7. Print the results, save the figure with the four plots and show it.
8. Stop.

D. PROGRAM

Python program (requires NumPy and Matplotlib). The defect categories must already be in descending order of count, as in the sample data:

```
# Experiment 12: Quality control tools - Pareto chart, histogram, scatter diagram, control chart
import numpy as np
import matplotlib.pyplot as plt

# (1) Defect counts for the Pareto chart - SAMPLE data
defects = {"Surface scratch": 38, "Dimension error": 27, "Burr": 18,
           "Porosity": 9, "Wrong finish": 5, "Other": 3}
# (2) Shaft diameters (mm), 30 readings in time order - SAMPLE data
dia = [25.001, 25.027, 25.024, 24.990, 24.994, 24.989, 25.011, 24.999, 25.015, 24.963,
       25.031, 24.998, 25.014, 24.997, 24.992, 25.009, 25.016, 24.996, 24.997, 25.014,
       24.983, 24.970, 25.008, 24.987, 24.962, 24.984, 24.991, 24.976, 24.970, 25.001]
# (3) Cutting speed (m/min) and surface roughness Ra (micrometre) - SAMPLE data
speed = [40, 60, 80, 100, 120, 140, 160, 180]
ra = [3.2, 2.9, 2.6, 2.2, 2.1, 1.8, 1.6, 1.5]

fig, ax = plt.subplots(2, 2, figsize=(11, 8))

# Pareto chart
names = list(defects.keys())
counts = list(defects.values())
total = sum(counts)
cum = np.cumsum(counts) / total * 100
ax[0, 0].bar(names, counts, color="tab:blue")
ax[0, 0].set_ylabel("Number of defects")
ax[0, 0].set_title("Pareto Chart")
ax[0, 0].tick_params(axis="x", rotation=30)
twin = ax[0, 0].twinx()
twin.plot(names, cum, color="red", marker="o")
twin.set_ylabel("Cumulative %")
twin.set_ylim(0, 105)
print("Pareto: cumulative % =", [round(float(c), 1) for c in cum])

# Histogram
mean = float(np.mean(dia))
sd = float(np.std(dia, ddof=1))
ax[0, 1].hist(dia, bins=6, color="tab:green", edgecolor="black")
ax[0, 1].axvline(mean, color="red", linestyle="--", label="Mean")
ax[0, 1].set_title("Histogram of Shaft Diameter")
ax[0, 1].set_xlabel("Diameter (mm)")
ax[0, 1].set_ylabel("Frequency")
ax[0, 1].legend()
print(f"Histogram: n = {len(dia)}, mean = {mean:.4f} mm, std dev = {sd:.4f} mm")
```

```
# Scatter diagram with straight-line fit
m, c = np.polyfit(speed, ra, 1)
r = np.corrcoef(speed, ra)[0, 1]
ax[1, 0].scatter(speed, ra, color="tab:purple", label="Data")
ax[1, 0].plot(speed, [m * s + c for s in speed], color="black", label="Best-fit line")
ax[1, 0].set_title("Scatter Diagram")
ax[1, 0].set_xlabel("Cutting speed (m/min)")
ax[1, 0].set_ylabel("Surface roughness Ra (micrometre)")
ax[1, 0].legend()
print(f"Scatter: Ra = {m:.4f} x speed + {c:.3f}, correlation r = {r:.3f}")

# Control chart for individual values (I chart), sigma from moving range
mr = [abs(dia[i] - dia[i - 1]) for i in range(1, len(dia))]
mrbar = sum(mr) / len(mr)
sigma = mrbar / 1.128                      # d2 = 1.128 for moving range of 2
ucl = mean + 3 * sigma
lcl = mean - 3 * sigma
ax[1, 1].plot(range(1, len(dia) + 1), dia, marker="o", color="tab:blue")
ax[1, 1].axhline(mean, color="green", label="CL")
ax[1, 1].axhline(ucl, color="red", linestyle="--", label="UCL")
ax[1, 1].axhline(lcl, color="red", linestyle="--", label="LCL")
ax[1, 1].set_title("Control Chart (Individual values)")
ax[1, 1].set_xlabel("Reading number")
ax[1, 1].set_ylabel("Diameter (mm)")
ax[1, 1].legend()
outside = [i + 1 for i, v in enumerate(dia) if v > ucl or v < lcl]
print(f"Control chart: CL = {mean:.4f}, UCL = {ucl:.4f}, LCL = {lcl:.4f}")
print("Points outside limits:", outside if outside else "none")

plt.tight_layout()
plt.savefig("qc_tools.png", dpi=150)
plt.show()
```

- **np.cumsum()** gives the running total for the cumulative percentage.
- **twinx()** adds a second y-axis for the cumulative line on the Pareto chart.
- **np.polyfit(x, y, 1)** gives the slope m and intercept c of the best-fit line; **np.corrcoef()** gives r.
- **plt.subplots(2, 2)** creates four plots in one figure.

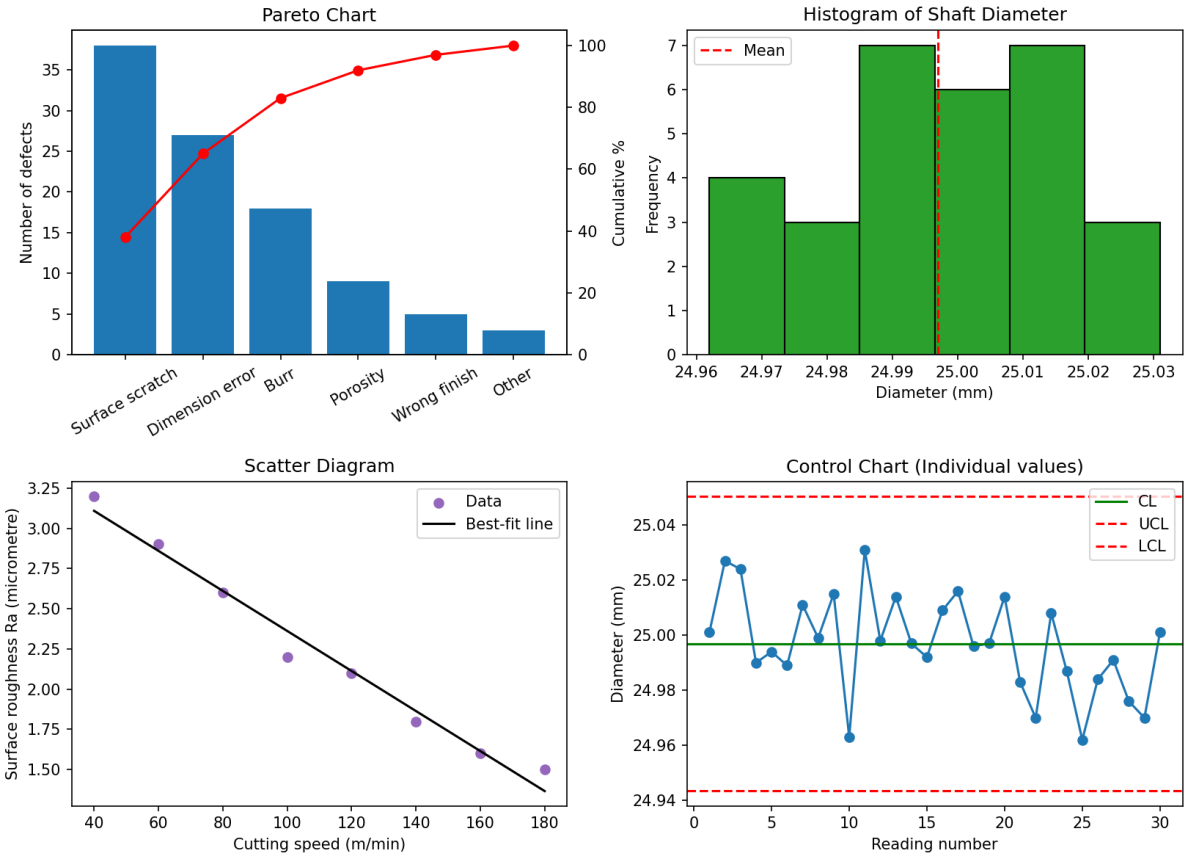
E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Pareto: cumulative % = [38.0, 65.0, 83.0, 92.0, 97.0, 100.0]
Histogram: n = 30, mean = 24.9970 mm, std dev = 0.0181 mm
Scatter: Ra = -0.0124 x speed + 3.606, correlation r = -0.989
Control chart: CL = 24.9970, UCL = 25.0505, LCL = 24.9434
Points outside limits: none
```

Expected graph: a 2 x 2 figure. (1) Pareto chart: six bars decreasing from 38 to 3 with a rising line reaching 100%. (2) Histogram of diameters centred near 25.00 mm. (3) Scatter diagram with a falling best-fit line (higher speed, lower roughness). (4) Control chart with readings scattered around the centre line, between UCL and LCL.

Generated figure (sample data):



F. OBSERVATION

Defect type	Count	Cumulative count	Cumulative %
Surface scratch	38	38	38.0
Dimension error	27	65	65.0
Burr	18	83	83.0
Porosity	9	92	92.0
Wrong finish	5	97	97.0
Other	3	100	100.0

Cutting speed (m/min)	40	60	80	100	120	140	160	180
Ra (micrometre)	3.2	2.9	2.6	2.2	2.1	1.8	1.6	1.5

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The Pareto chart, histogram, scatter diagram and control chart were plotted for the sample data. The top three defect types account for 83% of defects, the correlation between speed and roughness is -0.989, and no diameter reading was outside the control limits.

A. EXPERIMENT 13: STRESS DISTRIBUTION ALONG A PLATE (BAR)

B. AIM / OBJECTIVE

To write a Python program to plot the stress distribution along the length of a plate (bar) under axial load, for various lengths along the bar.

C. ALGORITHM

1. Start.
2. Enter the load P, thickness t and the segments (x start, x end, width).
3. For each segment, calculate the area A = w x t and the stress sigma = P / A.
4. Store the stress of the segment and print the segment table.
5. Print the stress at selected lengths x and the maximum stress.
6. Plot the plate widths and the stress against length x; save the figure.
7. Stop.

D. PROGRAM

Python program (requires Matplotlib):

```
# Experiment 13: Nominal axial stress along a stepped plate (bar) under tensile load
import matplotlib.pyplot as plt

P = 20000.0        # axial load in N (SAMPLE value)
t = 10.0           # plate thickness in mm (constant, SAMPLE value)
# (x_start mm, x_end mm, width mm) for each segment - SAMPLE geometry
segments = [(0, 100, 60.0), (100, 200, 40.0), (200, 300, 50.0)]

stress = []
print("Segment   x range (mm)   Width (mm)   Area (mm2)   Stress (MPa)")
for i, (x1, x2, w) in enumerate(segments, 1):
    area = w * t                # cross-section area in mm2
    sigma = P / area            # N/mm2 = MPa
    stress.append(sigma)
    print(f"{i:5d}     {x1:3d} - {x2:3d}      {w:8.1f}   {area:10.1f}   {sigma:10.2f}")


def stress_at(x):
    for (x1, x2, w), s in zip(segments, stress):
        if x1 <= x < x2:
            return s
    return stress[-1]


print("Stress at selected lengths:")
for x in [25, 75, 125, 175, 225, 275]:
    print(f"x = {x:3d} mm  ->  sigma = {stress_at(x):6.2f} MPa")
print(f"Maximum nominal stress = {max(stress):.2f} MPa in segment {stress.index(max(stress)) + 1}")

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 7), sharex=True)
for (x1, x2, w) in segments:
    ax1.fill_between([x1, x2], -w / 2, w / 2, color="lightsteelblue", edgecolor="black")
ax1.set_ylabel("Plate width (mm)")
ax1.set_title("Plate geometry (side view of widths)")
for (x1, x2, w), s in zip(segments, stress):
    ax2.plot([x1, x2], [s, s], color="tab:red", linewidth=2)
    ax2.text((x1 + x2) / 2, s + 1.5, f"{s:.2f} MPa", ha="center")
ax2.set_xlabel("Length along the plate, x (mm)")
ax2.set_ylabel("Nominal axial stress (MPa)")
ax2.set_title("Nominal stress distribution along the length")
ax2.set_ylim(0, max(stress) + 12)
plt.tight_layout()
plt.savefig("plate_stress.png", dpi=150)
```

```
plt.show()
```

- **segments** lists (x start, x end, width) for each part of the plate.
- **sigma = P / area** gives stress in MPa because P is in N and area in mm^2.
- **stress_at(x)** finds the segment that contains x and returns its stress.
- **fill_between** draws the plate outline; the red lines show stress in each segment.

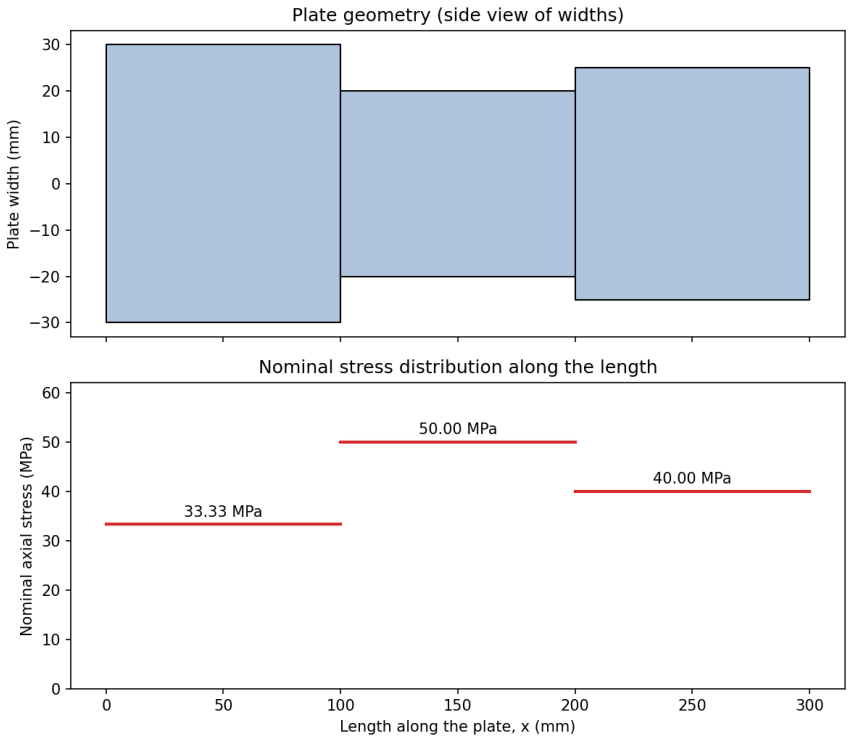
E. OUTPUT / EXPECTED OUTPUT

Output of the program above for the sample input (captured by running it with Python 3.12):

```
Segment   x range (mm)   Width (mm)   Area (mm2)   Stress (MPa)
    1       0 - 100          60.0        600.0        33.33
    2     100 - 200          40.0        400.0        50.00
    3     200 - 300          50.0        500.0        40.00
Stress at selected lengths:
x =  25 mm  ->  sigma =  33.33 MPa
x =  75 mm  ->  sigma =  33.33 MPa
x = 125 mm  ->  sigma =  50.00 MPa
x = 175 mm  ->  sigma =  50.00 MPa
x = 225 mm  ->  sigma =  40.00 MPa
x = 275 mm  ->  sigma =  40.00 MPa
Maximum nominal stress = 50.00 MPa in segment 2
```

Expected graph: the upper plot shows the plate widths in three steps. The lower plot shows three horizontal stress levels: 33.33 MPa for 0-100 mm, 50.00 MPa for 100-200 mm and 40.00 MPa for 200-300 mm, with jumps at the steps. Axes: x = length along the plate (mm), y = nominal axial stress (MPa). The sudden jumps are a simplification; real stress changes smoothly and peaks locally at the steps.

Generated figure (sample data):



F. OBSERVATION

Segment	x range (mm)	Width w (mm)	Area A (mm^2)	Nominal stress (MPa)
1	0 - 100	60	600	33.33
2	100 - 200	40	400	50.00
3	200 - 300	50	500	40.00

x (mm)	25	75	125	175	225	275
Stress (MPa)	33.33	33.33	50.00	50.00	40.00	40.00

All values marked 'sample' are illustrative, generated by running the program; they are not laboratory measurements.

G. RESULT

The nominal axial stress along the sample stepped plate was 33.33 MPa, 50.00 MPa and 40.00 MPa in the three segments, with a maximum of 50.00 MPa in the narrowest segment. Local stress concentrations at the steps were not calculated.